

CSC1029 OOProgramming

Lecture 8 – Managing Collections of Objects





What's to come in today's lecture?

- Arrays in Java
- Accessing array data
- Exception Handling

Array Declaration in Java ⁽¹⁾

- An **array** is defined by:
 - Its **name** (which starts with a lowercase letter)
 - The **type** of information it stores (primitive or object)
 - Its **size**

0	1	2	3	4	5	6	7	8	9
32	54	67	29	35	80	11	44	10	65

```
int scores[] = new int[10];
```

Array Declaration in Java (2)

- May be declared using the *new* operator

```
int scores[] = new int[10];
```

- Arrays are treated as objects : non-primitive

0	1	2	3	4	5	6	7	8	9
32	54	67	29	35	80	11	44	10	65

Array Declaration in Java ⁽³⁾

- May be declared and initialized at the same time:

```
int scores[] = {9,8,7,6,5,4,3,2,1,0};
```

- Size of the array is fixed by the list of values given – can't be modified after declaration

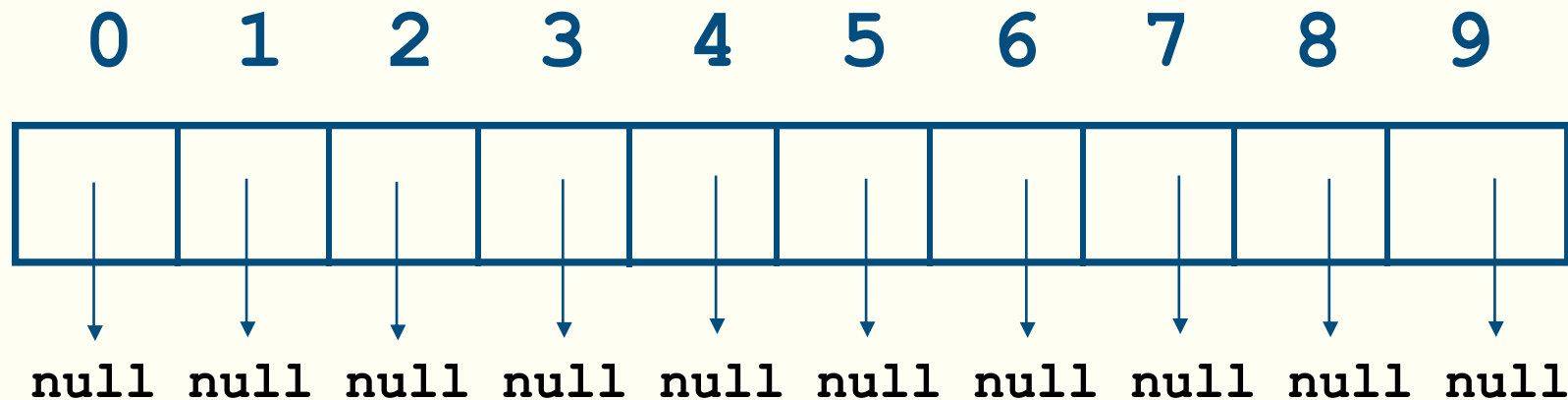
0	1	2	3	4	5	6	7	8	9
32	54	67	29	35	80	11	44	10	65

Array Declaration in Java (4)

- Arrays of non-primitives may be defined in the same way

```
String words[] = new String[10];
```

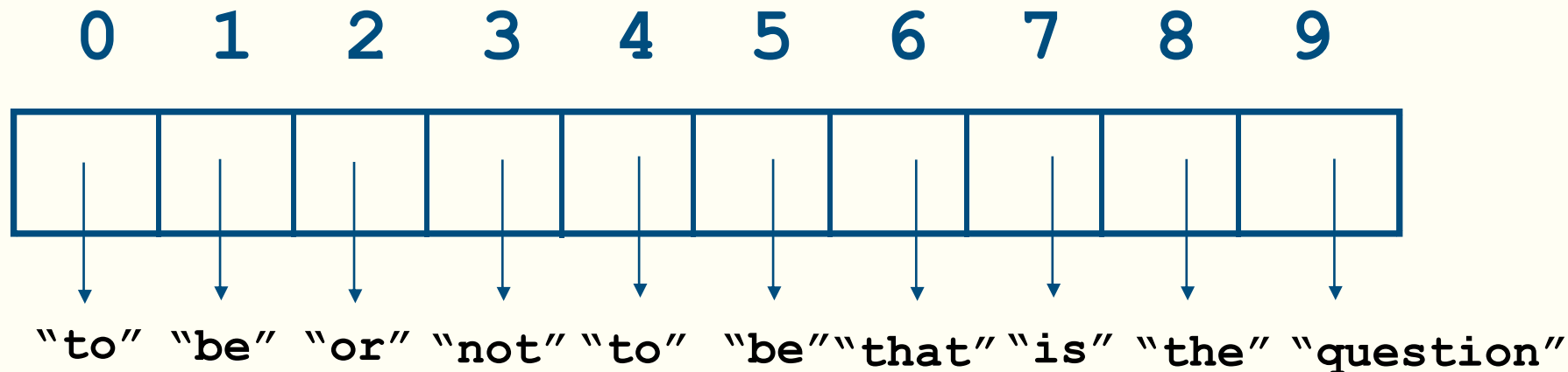
- This defines an array of String object references – **null** by default



Array Declaration in Java (5)

- Arrays of non-primitives may be defined in the same way

```
String words[] = {"to", "be", "or", "not", "to",  
                  "be", "that", "is", "the", "question"};
```



Accessing Array Data ⁽¹⁾

- Array data in Java is indexed from 0 upwards

0	1	2	3	4	5	6	7	8	9	
32	54	67	29	35	80	11	44	10	65	data

Data may be *read* from an array using an index value as follows:

```
int number = data[0] ; // read the 1st element
```

Accessing Array Data (2)

- Array data in Java is indexed from 0 upwards

0	1	2	3	4	5	6	7	8	9
32	54	67	29	35	80	11	44	10	65

data

Data may be *modified* in an array using an index value as follows:

```
data[6] = 4 ; // assign value 4 to element indexed by 6
```

Accessing Array Data (3)

- Array data in Java is indexed from 0 upwards

0	1	2	3	4	5	6	7	8	9
32	54	67	29	35	80	11	44	10	65

data

```
int size = data.length ; // size will have value 10
```

The *range of indices* for an array is *0 to len-1* (where *len* is the number of elements in the array)

Accessing Array Data (4)

Loops are commonly used in processing array data.

0	1	2	3	4	5	6	7	8	9	
32	54	67	29	35	80	11	44	10	65	data

```
int i = 0;
do{
    System.out.println(data[i]);
    i++;
}
while (i<data.length) ;
```

Do-While Loop

Accessing Array Data (5)

Loops are commonly used in processing array data.

0	1	2	3	4	5	6	7	8	9	
32	54	67	29	35	80	11	44	10	65	data

While Loop

```
int i = 0;
while (i < data.length) {
    System.out.println(data[i]);
    i++;
}
```

Accessing Array Data (6)

Loops are commonly used in processing array data.

0	1	2	3	4	5	6	7	8	9	
32	54	67	29	35	80	11	44	10	65	data

For Loop

```
for(int i=0;i<data.length;i++){  
    System.out.println(data[i]);  
}
```

Accessing Array Data ⁽⁷⁾

Loops are commonly used in processing array data.

0	1	2	3	4	5	6	7	8	9	
32	54	67	29	35	80	11	44	10	65	data

Enhanced For Loop

```
for(int value : data){  
    System.out.println(value) ;  
}
```

Accessing Array Data (8)

Loops are commonly used in processing array data.

0	1	2	3	4	5	6	7	8	9	
32	54	67	29	35	80	11	44	10	65	data

Consider the following:

```
System.out.println("For Loop");  
System.out.println("+-----+");  
for (i=0;i<data.length;i++) {  
    System.out.print(data[i] +" ");  
}
```

```
For Loop  
+-----+  
32 54 67 29 35 80  
11 44 10 65
```

What would the *output* look like?

Accessing Array Data (9)

Loops are commonly used in processing array data.

0	1	2	3	4	5	6	7	8	9	
32	54	67	29	35	80	11	44	10	65	data

Consider the following:

```
System.out.println("For Loop");  
System.out.println("+-----+");  
for (i=0; i<=data.length; i++) {  
    System.out.print(data[i] + " ");  
}
```

What happens if you try to access an element that does not exist?

Accessing Array Data (10)

Loops are commonly used in processing array data.

0	1	2	3	4	5	6	7	8	9
32	54	67	29	35	80	11	44	10	65

data

For Loop

+-----+

32 54 67 29 35 80 11 44 10 65

Exception in thread "main"

```
java.lang.ArrayIndexOutOfBoundsException: 10  
    at Week2.ArrayTest.main(ArrayTest.java:45)
```

Arrays as Method Parameters ⁽¹⁾

0 1 2 3 4 5 6 7 8 9

32	54	67	29	35	80	11	44	10	65
----	----	----	----	----	----	----	----	----	----

data (local to main)

```
public static void main(String args[]) {  
    int data[] = {32,54,67,29,35,80,11,44,10,65};  
    display(data);  
}
```

How do we make array **data** available to other methods?

Using parameters ... **reference** to data is **copied** to **display**. Array **elements** are **not copied**!

Arrays as Method Parameters ⁽²⁾

0 1 2 3 4 5 6 7 8 9

32	54	67	29	35	80	11	44	10	65
----	----	----	----	----	----	----	----	----	----

data (local to main)

```
public static void main(String args[]) {  
    int data[] = {32,54,67,29,35,80,11,44,10,65};  
    display(data);  
}  
    public static void display(int values[]) {  
        for(int i=0;i<values;i++){  
            System.out.print(values[i]+" ");  
        }  
    }  
}
```

ArraysAndMethods.java

Arrays as Method Parameters (3)

```
public static void main(String args[]) {  
    int data[] = {32,54,67,29,35,80,11,44,10,65};  
    display(data);  
    clear(data);  
    display(data);  
}  
  
public static void clear(int values[]) {  
    for(int i=0;i<values;i++){  
        values[i]=0;  
    }  
}
```

Arrays as Method Parameters (4)

So .. if we change values of an array parameter as part of a method execution.. it remains changed after the method completes .. ***so be careful!***

Console Output

0 0 0 0 0 0 0 0 0 0

ArraysAndMethods.java

The need for Exception Handling ⁽¹⁾

- *Exceptions* are circumstances that may arise in an executing program.
- Without specific management, an exception will cause a program to stop.
- There are many such circumstances that can arise.

For Loop

+-----+

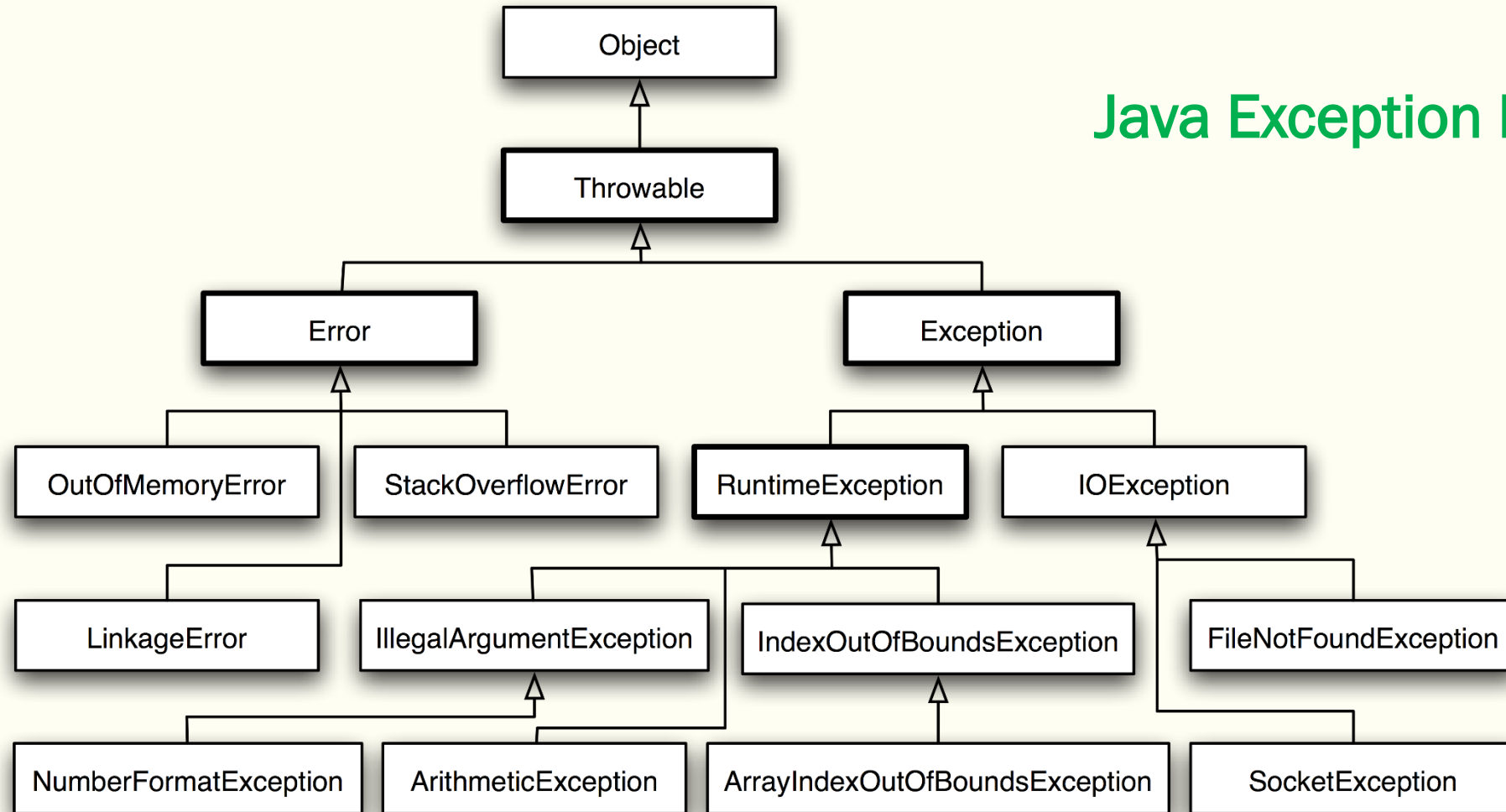
32 54 67 29 35 80 11 44 10 65

Exception in thread "main"

java.lang.ArrayIndexOutOfBoundsException: 10
at Week2.ArrayTest.main(ArrayTest.java:45)

The Need for Exception Handling (2)

Java Exception Hierarchy



The Need for Exception Handling (3)

```
public static void main(String[] args) {  
    Scanner input = new Scanner(System.in);  
  
    // input prompt - keep on same line - use 'print' use 'println'  
    System.out.print("Enter a double value:");  
    double dValue = input.nextDouble();  
    System.out.println("Double value:"+dValue);  
  
    input.close();  
}
```

Enter a double value:12.4
Double value:12.4

Enter a double value:hello
Exception in thread "main" [java.util.InputMismatchException](#)
at java.base/java.util.Scanner.throwFor([Scanner.java:939](#))
at java.base/java.util.Scanner.next([Scanner.java:1594](#))
at java.base/java.util.Scanner.nextDouble([Scanner.java:2564](#))
at week3.Exceptions.main([Exceptions.java:12](#))

The Need for Exception Handling (4)

```
public static void main(String[] args) {  
    Scanner input = new Scanner(System.in);  
  
    try {  
        // input prompt - keep on same line - use 'print' use 'println'  
        System.out.print("Enter a double value:");  
        double dValue = input.nextDouble();  
        System.out.println("Double value:"+dValue);  
    }  
    catch(Exception ex) {  
        System.out.println("Error - that value was not a double!");  
    }  
    input.close();  
}
```

Enter a double value:hello
Error - that value was not a double!

The Need for Exception Handling (5)

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    boolean validInput = false;
    do {
        try {
            System.out.print("Enter a double value:");
            double dValue = input.nextDouble();
            System.out.println("Double value:" + dValue);
            validInput = true;
        } catch (Exception ex) {
            System.out.println("Error - that value was not a double!");
            input.nextLine();
        }
    } while (!validInput);
    input.close();
}
```

Use a loop – repeat input process until user get's it right!

If input correct - change Boolean so that loop will terminate.

If input fails – display a message and clear ‘bad’ data from the Scanner



What's Next?

Case Study -

A system to manage a collection of objects

